

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Компьютерная Графика»
Тема: Исследование математических методов представления и
преобразования графических объектов на плоскости и в пространстве

Студенты гр. 3311	_____	Землякова С. А.
	_____	Кудрин П. В.
	_____	Сапронов К. Д.
Преподаватель	_____	Колев Г.Ю.

Цель работы

Отображение плоского объекта относительно осей координат с изменением или без изменения масштаба изображения.

Теоретическая часть

В ходе выполнения лабораторной работы реализовано математическое представление и преобразование графических объектов на плоскости с использованием аффинных преобразований. Данные преобразования применяются для изменения масштаба изображения, отражения объектов относительно координатных осей и поворота на произвольный угол.

Математические основы преобразований

Для работы с двумерными объектами используется матричный аппарат аффинных преобразований. Каждая точка объекта представляется в однородных координатах $[x, y, 1]$, что позволяет применять линейные преобразования с помощью матриц 3×3 .

Матрицы преобразований, используемые в работе:

1. Отражение относительно оси X:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

2. Отражение относительно оси Y:

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

3. Равномерное масштабирование:

$$\begin{bmatrix} s & 0 & 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 & s & 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

4. Неравномерное масштабирование:

$$\begin{bmatrix} s_x & 0 & 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 & s_y & 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

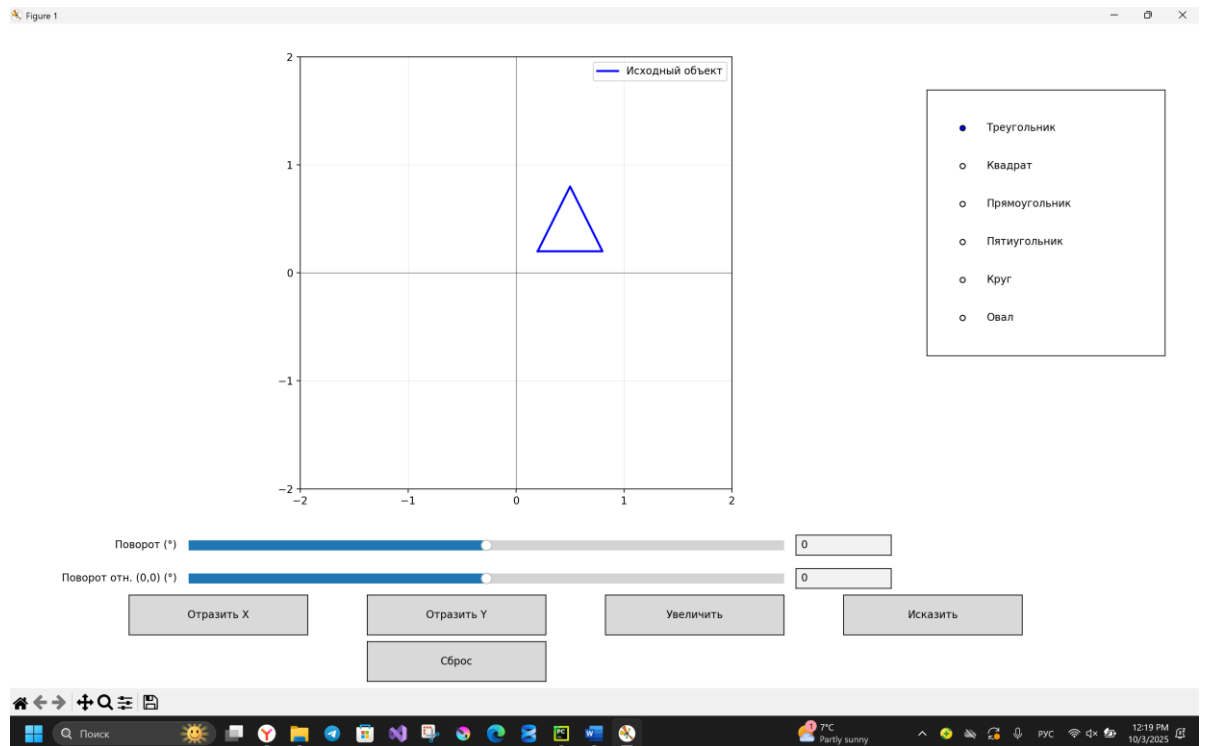
5. Поворот на угол θ :

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \end{bmatrix}$$
$$\begin{bmatrix} \sin\theta & \cos\theta & 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

Особенности реализации

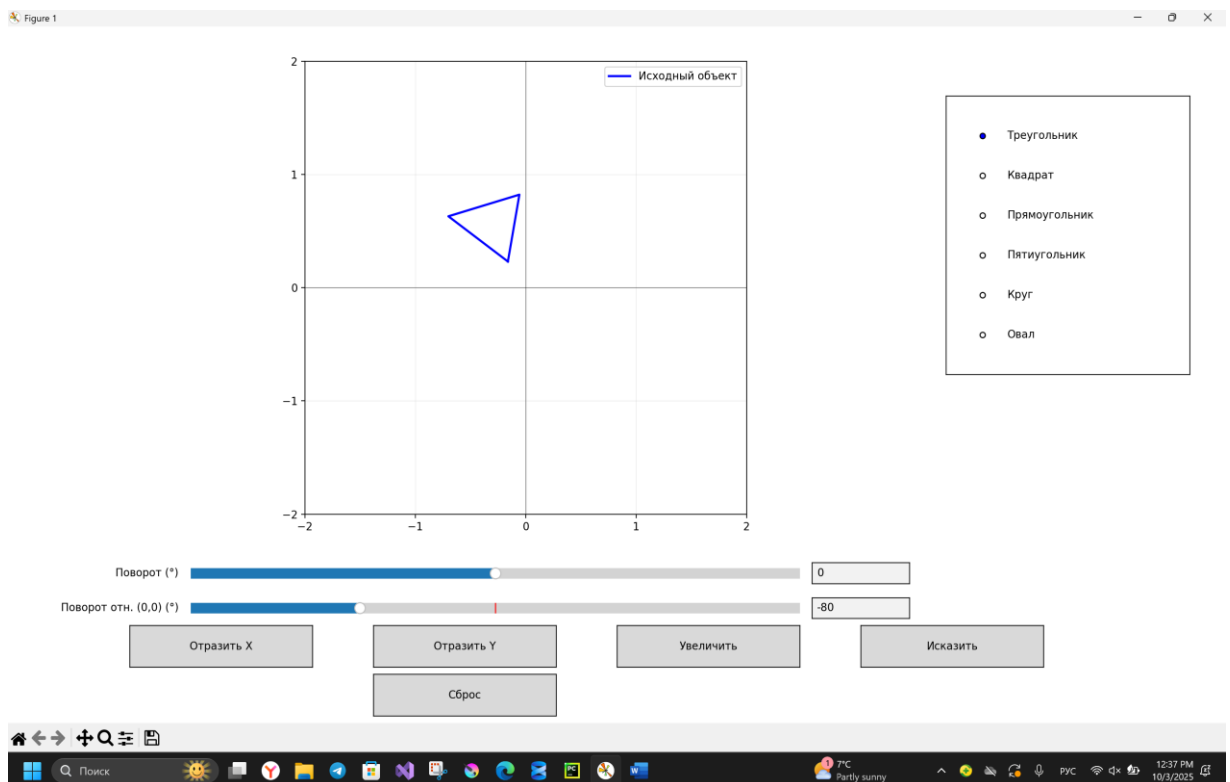
При запуске программы пользователю предоставляется интерактивный интерфейс с следующими возможностями:

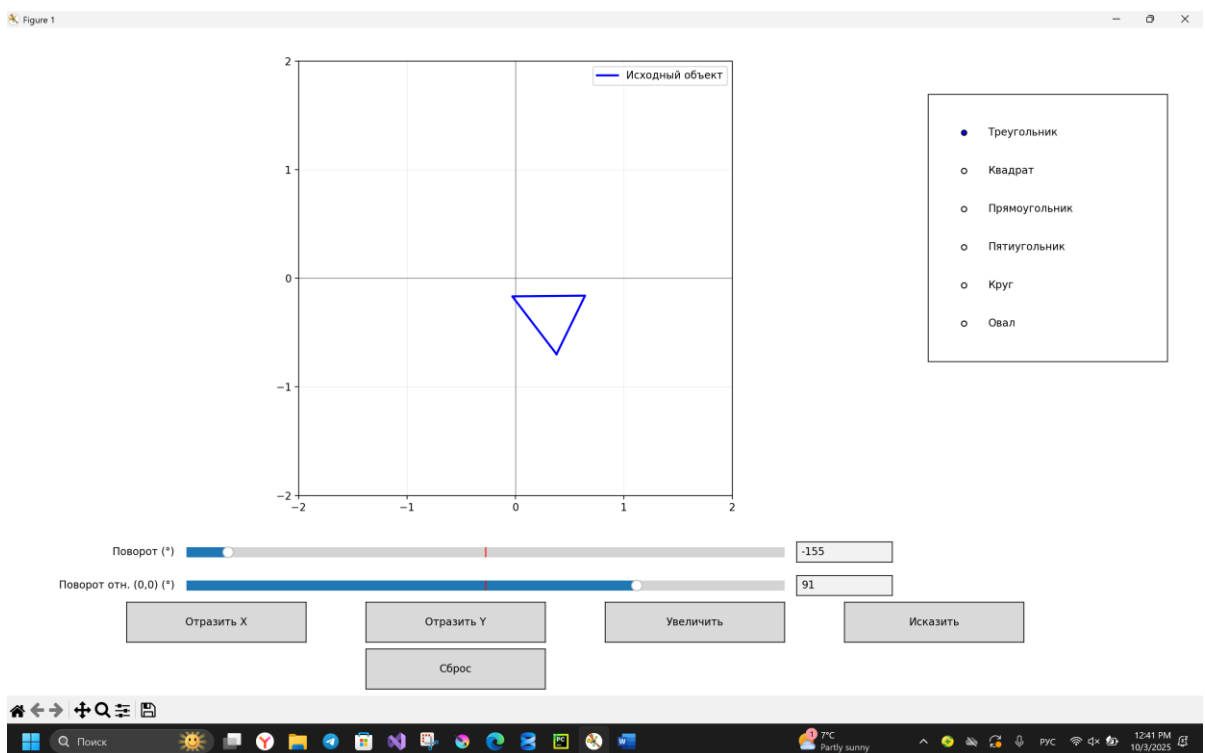
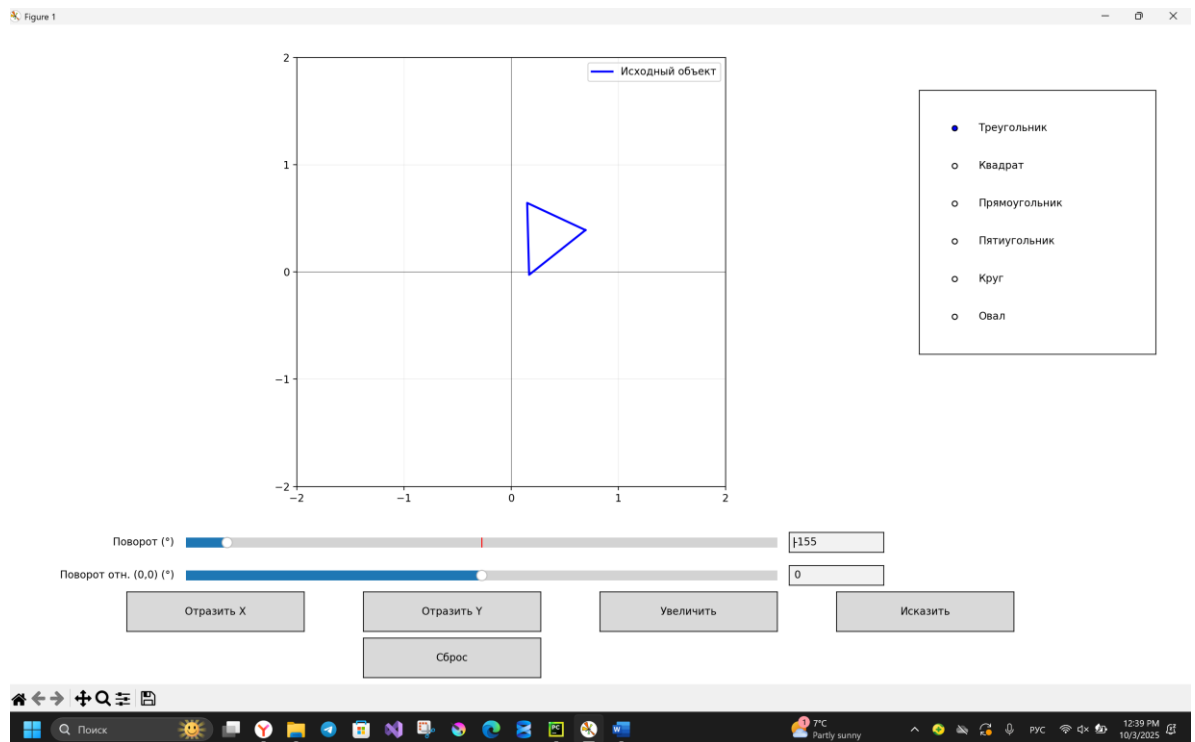
1. Выбор фигур - доступны 6 стандартных фигур: треугольник, квадрат, прямоугольник, пятиугольник, круг, овал



2. Система поворотов:

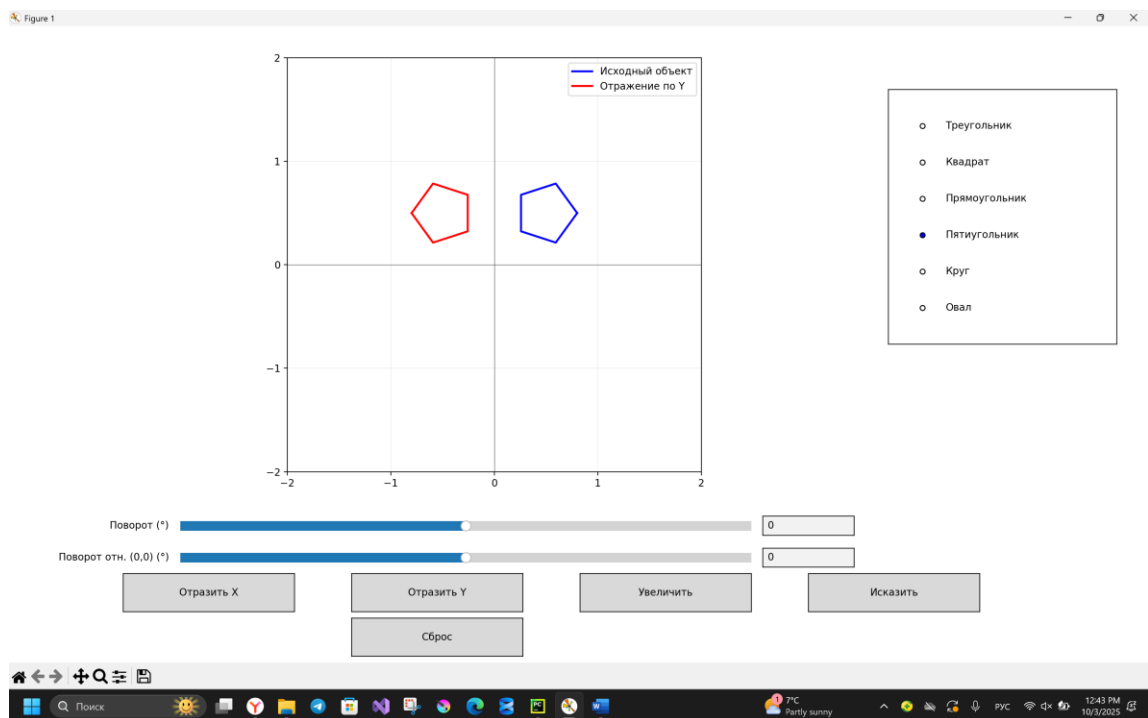
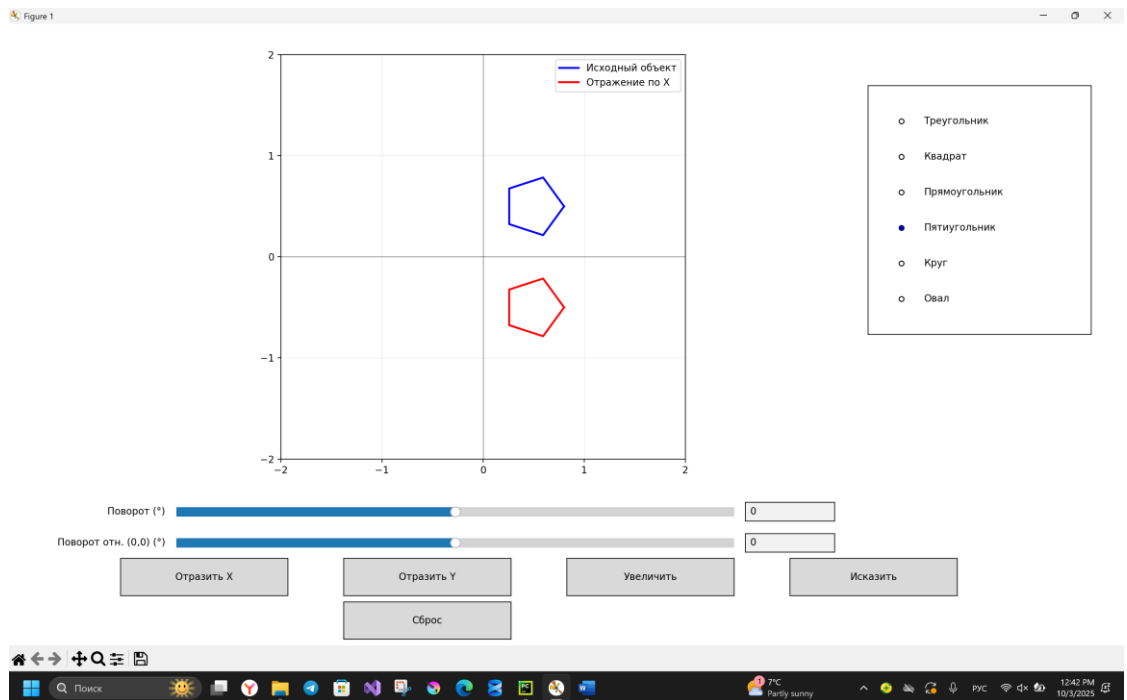
- Поворот относительно центра фигуры
- Поворот относительно начала координат (0, 0)
- Оба поворота могут комбинироваться

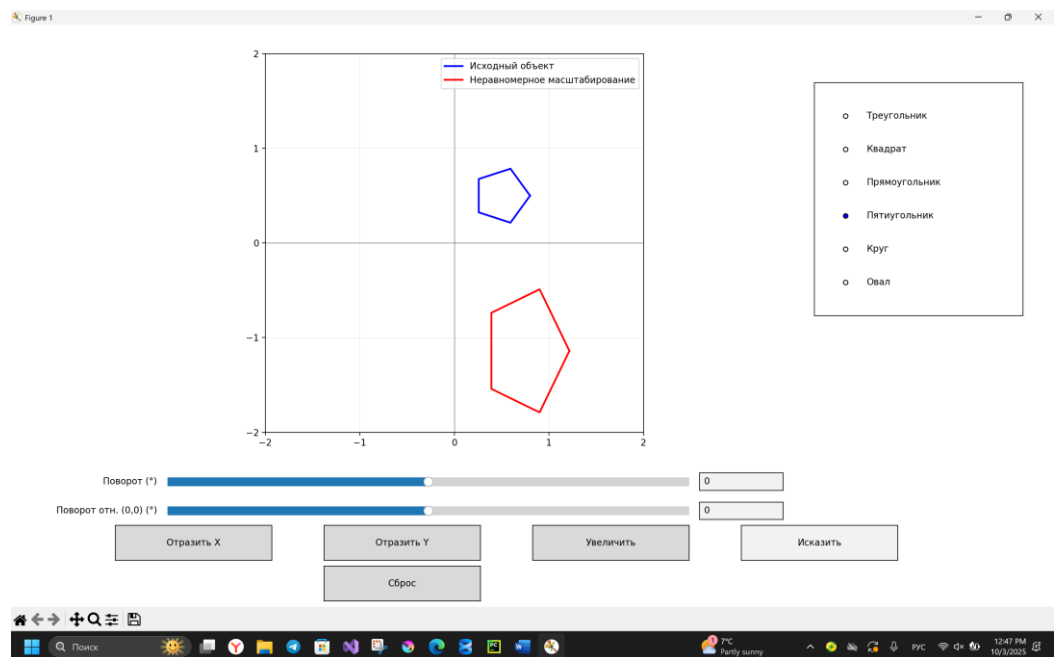
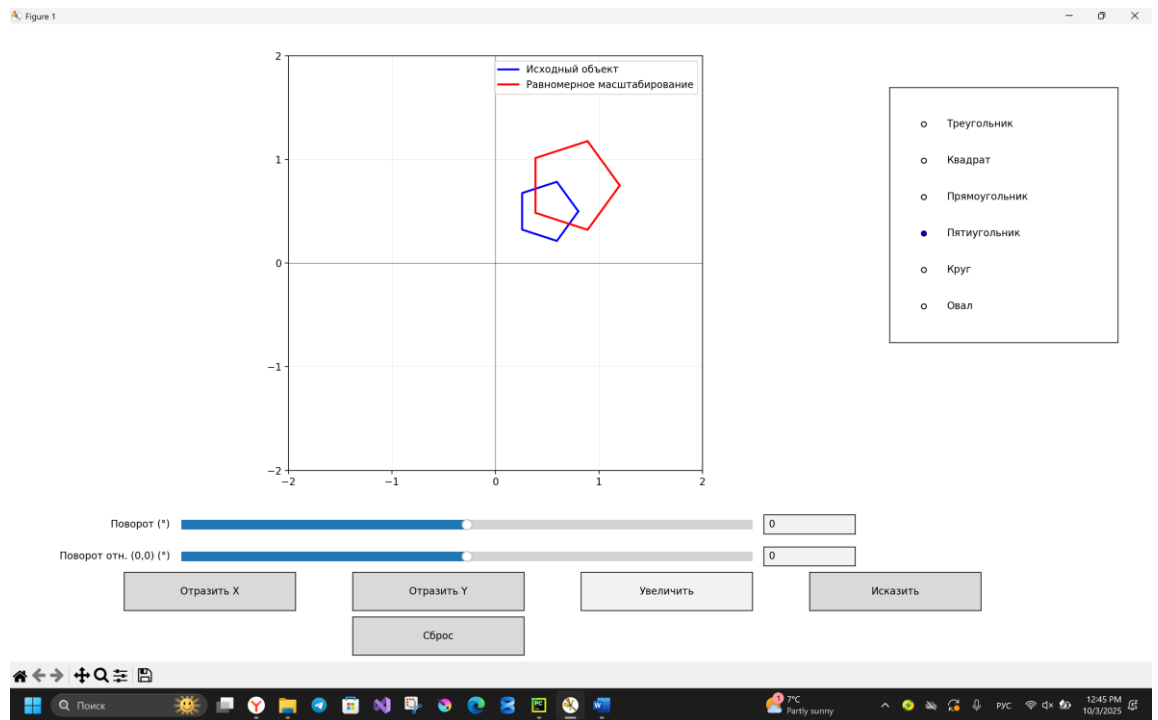




3. Преобразования:

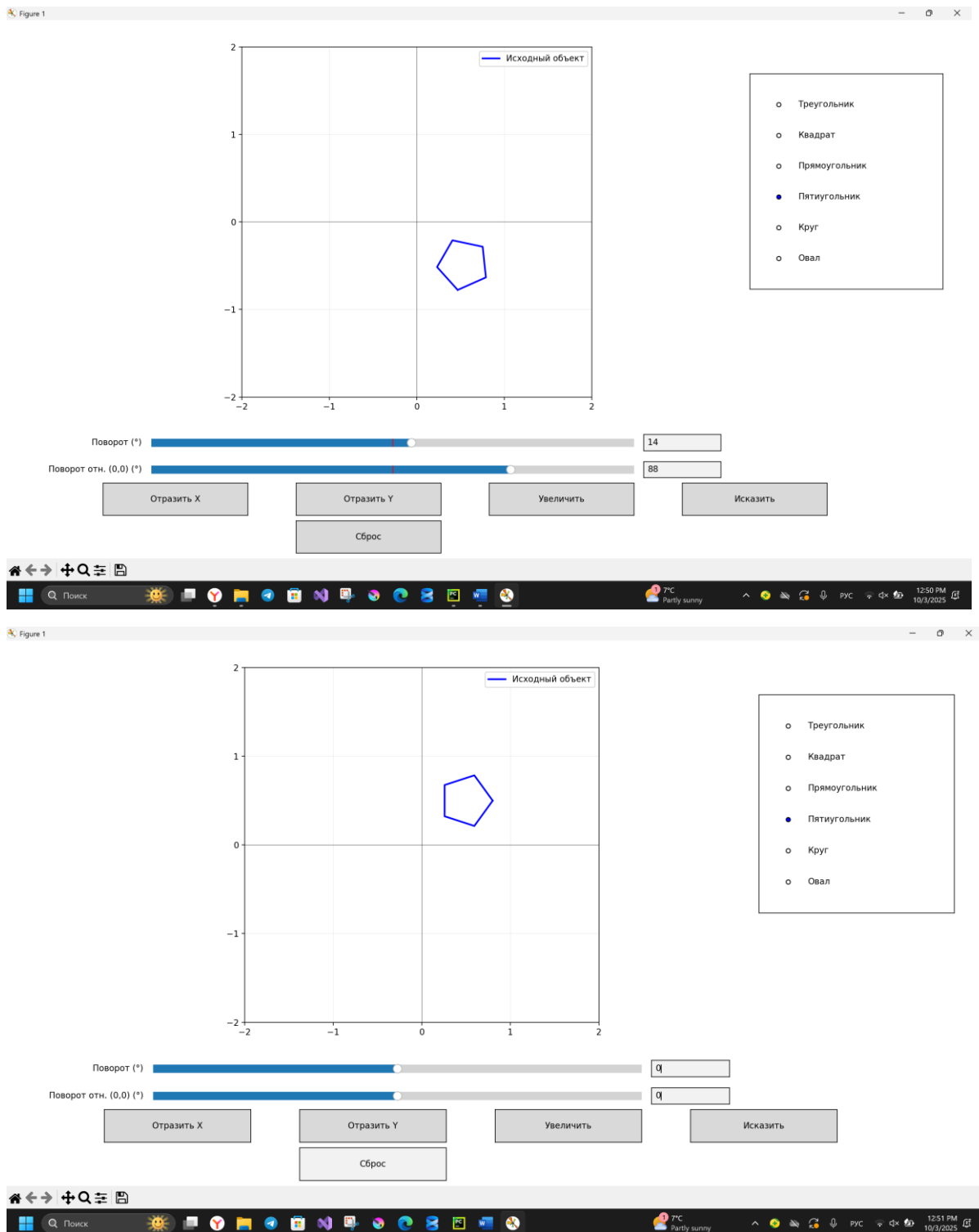
- Отражение относительно осей OX и OY
- Равномерное масштабирование (увеличение)
- Неравномерное масштабирование (искажение)





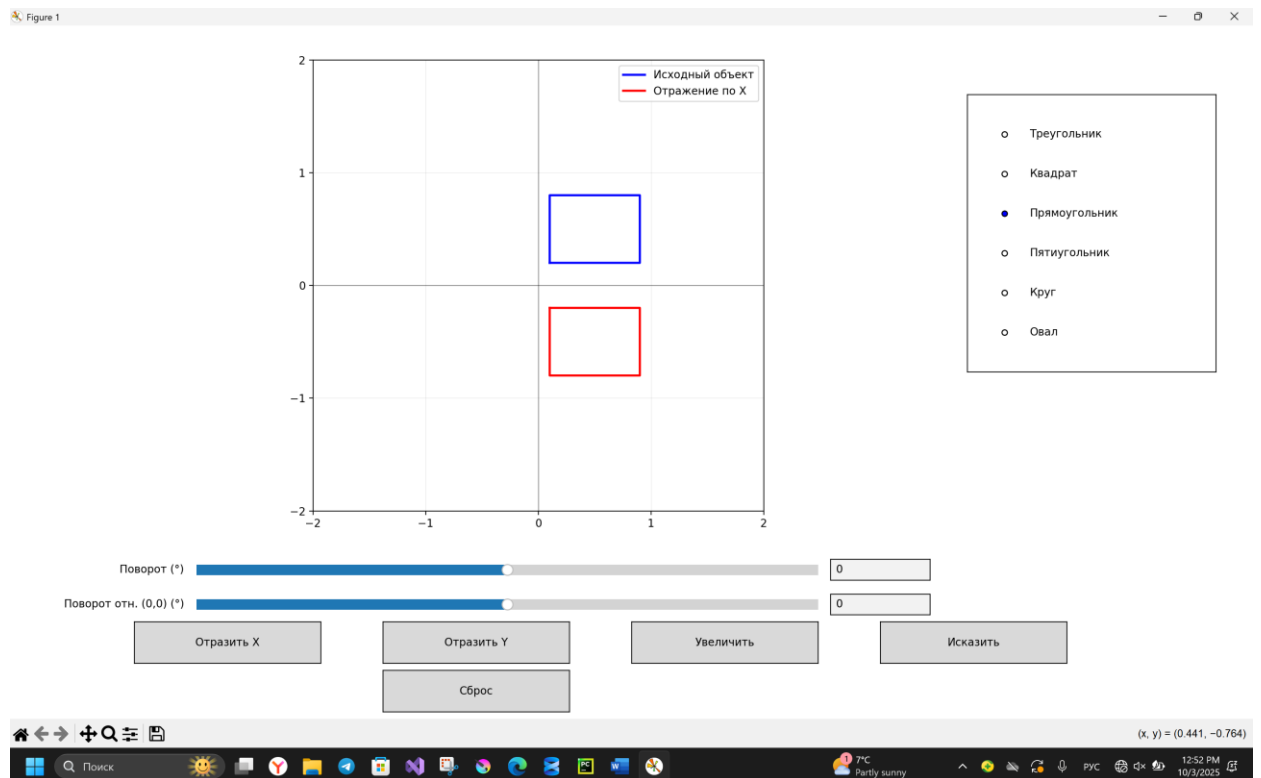
4. Интерактивные элементы управления:

- Ползунки для плавного изменения углов поворота
- Текстовые поля для точного ввода углов
- Кнопка "Сброс" для возврата к исходному состоянию



5. Визуализация:

- Декартова система координат с осями X и Y
- Мелкая сетка с шагом 0.2 для точного позиционирования
- Подписанные деления по осям
- Исходный объект отображается синим цветом
- Преобразованный объект отображается красным цветом



Технические особенности:

- Реализация на Python с использованием библиотек NumPy и Matplotlib
- Объектно-ориентированный подход для удобства управления состоянием
- Матричные преобразования применяются к однородным координатам
- Все преобразования комбинируются корректно благодаря последовательному применению матриц

Вывод

В рамках данной лабораторной работы была успешно разработана и реализована программа для выполнения различных преобразований графических объектов на плоскости. Созданное приложение предоставляет пользователю интуитивно понятный интерфейс для работы с геометрическими фигурами, включая такие операции как отражение относительно координатных осей, масштабирование с возможностью как равномерного увеличения, так и асимметричного искажения, а также поворот вокруг двух различных центров - как относительно собственного центра фигуры, так и относительно начала системы координат.

Исходный код

```
def apply_rotation(self, points):
    # применяет поворот к точкам относительно центра
    if self.rotation_angle == 0:
        return points

    angle_rad = np.radians(self.rotation_angle)
    cos_a, sin_a = np.cos(angle_rad), np.sin(angle_rad)

    # центр фигуры
    center = np.mean(points, axis=0)

    # поворот относительно центра
    rotated = points.copy()
    rotated -= center
    rotated = np.dot(rotated, np.array([[cos_a, -sin_a], [sin_a, cos_a]]))
    rotated += center

    return rotated
```

```
def apply_origin_rotation(self, points):
    # применяет поворот к точкам относительно начала координат (0,0)
    if self.origin_rotation_angle == 0:
        return points

    angle_rad = np.radians(self.origin_rotation_angle)
    cos_a, sin_a = np.cos(angle_rad), np.sin(angle_rad)

    # поворот относительно начала координат
    rotated = points.copy()
    rotated = np.dot(rotated, np.array([[cos_a, -sin_a], [sin_a, cos_a]]))

    return rotated
```

```
def apply_all_rotations(self, points):
    # применяет оба поворота: сначала относительно начала координат, потом
    # относительно центра
    points = self.apply_origin_rotation(points)
    points = self.apply_rotation(points)
    return points

def apply_transform(self, points, transform_matrix):
    # применяет аффинное преобразование
    homogeneous_points = np.hstack([points, np.ones((points.shape[0], 1))])
    transformed_points = homogeneous_points @ transform_matrix.T
    return transformed_points[:, :2]
```

```
def on_reflect_x(self, event):
    original = self.get_current_object()
    rotated_original = self.apply_all_rotations(original) # повернутая
    # исходная фигура

    transform = np.array([[1, 0, 0], [0, -1, 0], [0, 0, 1]])
    transformed = self.apply_transform(rotated_original, transform) #
    # отражение повернутой

    self.redraw_transformed(transformed, 'Отражение по X')
```

```
def on_reflect_y(self, event):
    original = self.get_current_object()
    rotated_original = self.apply_all_rotations(original)

    transform = np.array([[-1, 0, 0], [0, 1, 0], [0, 0, 1]])
    transformed = self.apply_transform(rotated_original, transform)

    self.redraw_transformed(transformed, 'Отражение по Y')
```

```
def on_scale_uniform(self, event):
    # равномерное масштабирование
    original = self.get_current_object()
    transform = np.array([[1.5, 0, 0], [0, 1.5, 0], [0, 0, 1]])
    transformed = self.apply_transform(original, transform)
    self.redraw(transformed, 'Равномерное масштабирование')

def on_scale_non_uniform(self, event):
    # неравномерное масштабирование
    original = self.get_current_object()
    transform = np.array([[1.52, 0, 0], [0, -2.28, 0], [0, 0, 1]])
    transformed = self.apply_transform(original, transform)
    self.redraw(transformed, 'Неравномерное масштабирование')
```